Vivo Clienteling CRM

End-to-End Technical Architecture

Vivo Fashion Group — Kenya · Uganda · Rwanda

Built on the Emergent platform

FastAPI · React 19 · MongoDB Atlas

Document version: May 2026

Production: https://crmvivofashiongroup.com

Vivo Clienteling CRM — End-to-End Technical Document

> Project: Vivo Fashion Group Clienteling CRM (v1) Owner: Vivo Fashion Group (Kenya / Uganda / Rwanda) Built on: Emergent platform · FastAPI + React + MongoDB Atlas Production: https://crmvivofashiongroup.com Document version: May 2026

Table of Contents

1. Executive Summary

The Vivo Clienteling CRM is a tablet-first web application for in-store sales associates at Vivo Fashion Group. It transforms ~163,000 historical customer records from the company's BigQuery-backed BI system into actionable, real-time clienteling tools: customer 360 profiles, AI-suggested next actions, WhatsApp/SMS messaging, personalised lookbooks, loyalty management, and manager-level KPI dashboards.

Headline numbers (May 2026):

Metric	Value
Customers under management	163,369
Rolling 12-mo revenue tracked	KES 1,119,003,054
Active loyalty members	62,067 (Bronze 60,968 / Silver 918 / Gold 99 + Dormant 101,384)
Backend endpoints	~150 (`/api/*`)
Lines of Python	~3,400 (server.py) + 600 (loyalty.py) + 200 (per-module routes)
Lines of React	~30,000 across 50+ pages/components
Real BI orders processed per year	~280,000 line items
Customer cache full sync time	~6 min

2. Problem Statement & Business Goals

Original brief

> *"Build CRM"* — followed by the Vivo Fashion Group CRM brief specifying a tablet-first clienteling app (NOT a full enterprise CRM) for in-store associates in Kenya, Uganda, and Rwanda, plus the Shop Zetu online channel.

User personas

Persona	Needs
Associate (in-store sales staff)	Fast customer lookup, profile context, follow-ups, send messages, build lookbooks during the shift.
Manager (store/regional/HQ)	Sales KPIs from BI, per-associate activity, top customers, churn-risk lists, social listening, message-template admin, audit log for Kenya DPA compliance.
Member (loyalty customer)	Mobile app showing tier, points/spend progress, vouchers, anniversary status.

Business goals (per brief Section 6.1)

- ✅ Lift average customer lifetime value via targeted clienteling
- ✅ Reduce churn (180+ day inactivity) via proactive associate outreach
- ✅ Track WhatsApp/SMS conversion to revenue (clienteling attribution KPI)
- ✅ Compliance with Kenya Data Protection Act (DPA): consent, audit, "right to be forgotten"
- ❌ NOT in v1: POS/checkout, inventory editing, bulk-marketing, ticketing, B2B pipeline, loyalty rules engine (now in v1.5+)

---

3. Technology Stack

Backend

- Python 3.11 + FastAPI (async, all routes under `/api/`)
- Motor (async MongoDB driver) + PyMongo for bulk ops
- APScheduler (in-process cron for nightly syncs, FB pulls, loyalty refresh)
- httpx (async HTTP client for BI + Facebook Graph API)
- emergentintegrations (Emergent's LLM key wrapper for Claude / GPT / Gemini)
- bcrypt + python-jose for session signing
- firebase-admin (FCM push, lazy-loaded, off by default)

Frontend

- React 19 + Create React App (CRACO) for build
- Tailwind CSS + shadcn/ui component library (`/components/ui/`)
- lucide-react icons (NO emoji icons per design guidelines)
- recharts for charts
- axios (configured with credentials for cookie-based auth)

- sonner for toasts
- react-router-dom v7

Mobile

- React Native + Expo SDK 51 + expo-router
- OTP-based sign-in (mock provider in dev — passes `000000`)

Database

- Local dev / preview: Local MongoDB via supervisor
- Production: MongoDB Atlas cluster (`cluster0.vum5nmb.mongodb.net`, DB `vivo_crm`)
- 25+ collections (full schema in §6)

External services

- Vivo BI API (BigQuery-backed read-only feed) — base: `https://vivo-bi-api-666430550422.europe-west1.run.app`
- Emergent Managed Google OAuth for sign-in
- Facebook Graph API v19 for social listening
- Claude Sonnet 4.5 via Emergent LLM key for NBA / sentiment / brief generation

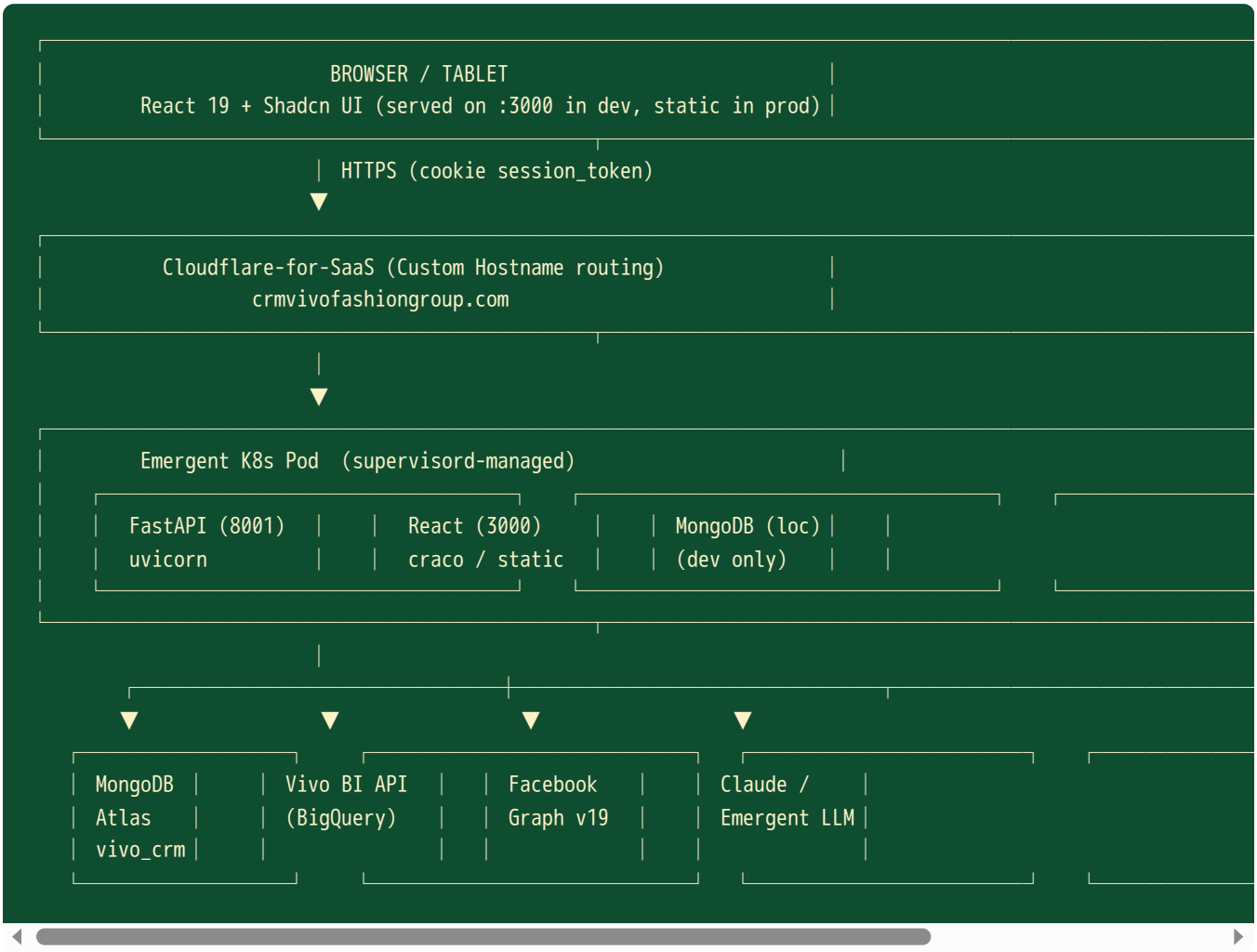
Infrastructure

- Emergent native deployment (Kubernetes containers + Cloudflare-for-SaaS custom hostname)
- supervisord for process management (backend / frontend / mongodb / expo)

---

4. System Architecture

High-level diagram



Request flow — typical "open customer profile"

1. Associate clicks customer in search results → React Router navigates to `/customers/:id`
2. Component mounts → fires 6 parallel API calls:
3. `/api/bi/customer/{id}` → profile + lifetime aggregates
4. `/api/customers/{id}/notes` → notes timeline
5. `/api/customers/{id}/tasks` → follow-ups
6. `/api/customers/{id}/messages` → SMS/WhatsApp history
7. `/api/customers/{id}/preferences` → style preferences
8. `/api/loyalty/customer/{id}` → tier + voucher + progress
9. Backend resolves each call:
10. BI calls go through 60s in-memory cache → BI API (with 3× retry, 503-backoff)
11. Mongo calls use Motor async with projection (exclude `_id`)

12. NBA card lazy-loads from `/api/customers/{id}/nba` (6h cache) — fires Claude only on cache miss

Key architectural decisions

Decision	Rationale
Read-only BI, CRM doesn't write back to BigQuery	Keeps Vivo's data ownership clean — CRM is purely a UI/insight layer over the truth source
In-memory 60s BI cache	Prevents hammering BI on every page load; safe because BI itself updates every ~10 min
Customer-cache mirror in Mongo	Required for pagination, filtering, RFM tagging, and offline-style queries — BI doesn't support these
All routes prefixed `/api`	K8s ingress strips `/api` and routes to FastAPI on `:8001`
Sessions via httpOnly cookie + Bearer header fallback	Cookie path works for browser; Bearer header lets testing agents script directly
APScheduler in-process (not Celery/RQ)	Single replica deployment — no need for an external queue; nightly/15-min jobs are light
`@app.on_event("startup")` schedulers	Run alongside FastAPI lifespan; deferred (60-120s) so K8s readiness probe passes first

5. Repository Layout

```
/app/
├── backend/
│   ├── server.py                    # Main FastAPI app, 3,400 lines
│   ├── loyalty.py                   # Loyalty engine (tiers, vouchers, refresh)
│   ├── sms.py                       # Mock SMS/WhatsApp provider
│   ├── facebook_sync.py             # FB Graph → social_posts/social_feedback
│   ├── fcm_worker.py                # Mobile push (FCM, off by default)
│   ├── requirements.txt
│   ├── .env                         # MONGO_URL, DB_NAME, EMERGENT_LLM_KEY, …
│   │
│   ├── routes/                      # Sub-routers wired into server.py
│   │   ├── duplicates.py             # Phone-only duplicate scan (cached)
│   │   ├── customer_sync.py          # (Endpoints inlined into server.py)
│   │   ├── whatsapp.py               # WhatsApp BSP scaffold (mock)
│   │   ├── social.py                 # 11 social-listening endpoints
│   │   ├── insights.py               # Manager dashboards, cohort analytics
│   │   ├── training.py               # Associate training mode (sandbox)
│   │   └── push.py                   # Mobile push notification composer
│   │
│   ├── scripts/                     # One-off analytics + cleanup scripts
│   │   ├── simulate_2025_loyalty.py
│   │   ├── simulate_2025_vouchers.py
│   │   ├── sim_flat_points_2025.py
│   │   ├── sim_hybrid_points_2025.py
│   │   ├── sim_hybrid_v2_2025.py
│   │   ├── sim_hybrid_v3_12mo.py
│   │   ├── validate_below_floor.py
│   │   ├── audit_walkins.py
│   │   ├── purge_walkins.py
│   │   └── tier_threshold_and_points_cost.py
│   │
│   └── tests/                       # pytest backend regression suite
│
├── frontend/
│   ├── src/
│   │   ├── pages/                    # 50+ pages
│   │   │   ├── Today.jsx             # Associate home (call list, KPIs)
│   │   │   ├── CustomerDatabase.jsx # Paginated 163k customer grid
│   │   │   ├── CustomerProfile.jsx  # 360 customer view
│   │   │   ├── Loyalty.jsx           # Tier dashboard + drill-downs
│   │   │   ├── Inbox.jsx             # Social feedback / DM triage
│   │   │   ├── Insights.jsx          # Manager KPI dashboards
│   │   │   ├── Cohorts.jsx           # Cohort retention analysis
│   │   │   ├── Operations.jsx        # Audit log, system status
│   │   │   ├── Settings.jsx          # Templates, FB pages, sync controls
│   │   │   ├── Lookbooks.jsx         # Drag-build personalised lookbooks
│   │   │   ├── PublicLookbook.jsx    # Shareable URL view (no auth)
│   │   │   └── …
│   │   ├── components/               # 30+ reusable components
│   │   │   ├── ui/                   # Shadcn primitives
│   │   │   ├── AppShell.jsx
│   │   │   ├── TierCustomersModal.jsx
```

```
│   │   │   ├── VoucherCostPanel.jsx
│   │   │   ├── FreshnessBadge.jsx
│   │   │   ├── LoyaltyBadge.jsx
│   │   │   └── ...
│   │   └── lib/api.js              # Axios wrapper with REACT_APP_BACKEND_URL
│   ├── package.json
│   ├── tailwind.config.js
│   └── .env                        # REACT_APP_BACKEND_URL only
│
├── mobile/                         # React Native / Expo app
│   ├── app/                        # expo-router pages
│   ├── components/
│   ├── lib/
│   ├── app.json                    # Expo config (triggers supervisor entry)
│   ├── package.json
│   └── .env                        # EXPO_PUBLIC_BACKEND_URL, ...
│
├── odoo/                           # Odoo 18 POS connector (future)
│
├── memory/
│   ├── PRD.md                      # Product spec + changelog
│   ├── VIVO_CRM_ARCHITECTURE.md    # This file
│   └── test_credentials.md         # Test sessions for QA bypass
│
└── test_reports/                   # Iteration JSON reports from testing agent
```

6. Database Schema

All collections in MongoDB Atlas DB `vivo_crm`. Mongo `_id` is always excluded from API responses.

Identity & sessions

```
users
  user_id (str, PK)         e.g. usr_1741200000000
  email (str)
  name (str)
  role (str)                'manager' | 'associate'
  active (bool)
  created_at (ISO str)
  last_login_at (ISO str)

user_sessions
  session_token (str, PK)   signed opaque token
  user_id (str)
  created_at, expires_at (ISO str)
```

Customer cache (the heart of the app)

```
customer_cache
  customer_id (str, PK)           BI customer_id
  customer_name (str)
  phone (str | null)
  email (str | null)
  city (str | null)
  customer_country (str)          'Kenya' | 'Uganda' | 'Rwanda'
  total_orders (int)              lifetime
  total_units (int)               lifetime
  total_sales (float)             lifetime KES
  avg_basket (float)
  first_purchase_date (str ISO)
  last_purchase_date (str ISO)
  rfm_tier (str)                  'vip'|'loyal'|'promising'|'at_risk'|'churned'|'new'
  loyalty_tier (str)              'dormant'|'bronze'|'silver'|'gold'
  spend_12mo_kes (float)          rolling 12-mo (refreshed nightly)
  loyalty_recomputed_at (ISO str)
  cached_at (ISO str)

  Indexes:
    customer_id (unique)
    last_purchase_date
    loyalty_tier
    customer_country
```

Clienteling — owned by CRM

```
notes
  note_id, customer_id, author_user_id, body, created_at

tasks
  task_id, customer_id, assignee_user_id, due_date, status,
  title, body, created_by, created_at, completed_at

customer_preferences
  customer_id (PK), sizes [], fits [], fabrics [], occasions [],
  brand_affinities [], updated_by, updated_at

messages
  message_id, customer_id, channel ('whatsapp'|'sms'),
  body, template_id, sender_user_id, delivery_status, sent_at

consent
  customer_id (PK), whatsapp (bool), sms (bool), email (bool),
  updated_at, history [{channel, opted_in, at, by}]

audit_log
  event_id, actor_user_id, action, entity_type, entity_id,
  details, ip, ua, at
```

Lookbooks

```
lookbooks
  lookbook_id, customer_id, title, items [{sku, image_url, note}],
  associate_user_id, share_token (UUID), created_at, expires_at

lookbook_interests   (from public share)
  lookbook_id, sku, customer_id, at, ip
```

Loyalty

```
loyalty_config            single doc, _id='singleton'
  qualify: {silver, gold}              KES thresholds
  retain: {silver, gold}
  voucher_kes: {bronze, silver, gold}
  voucher_validity_days
  grace_period_days
  discount_pct: {bronze, silver, gold}
  …

loyalty_vouchers
  voucher_id, customer_id, amount_kes, tier_at_issue,
  reason, status ('issued'|'redeemed'|'expired'),
  issued_at, expires_at, redeemed_at

loyalty_styling          Gold annual session bookings
  session_id, customer_id, scheduled_for, stylist_name, notes, …

loyalty_audit
  audit_id, action, customer_id, actor_user_id, data, at
```

Social listening

```
social_posts             every brand-page post (Facebook etc.)
social_feedback          comments, reviews, DMs
social_handles           customer ↔ handle linking
social_replies
auto_tasks_runs          weekly quality auto-task generator log
```

Mobile push

```
push_devices             {customer_id, fcm_token, platform, opted_in}
push_outbox              {push_id, title, body, segment, status, scheduled_at}
```

Sync state

```
sync_jobs
  _id (job_id, e.g. 'customer_cache_full_sync')
  status ('running'|'completed'|'error')
  started_at, updated_at, finished_at
  current_range, completed_quarters, customers_in_cache, …

duplicates_cache         {_id: 'latest', groups [], scanned_at, …}
```

Lookbook product catalogue (cached from BI)

```
product_catalog
  sku (PK), title, brand, collection, subcategory, color, size,
  image_url, list_price_kes, updated_at
```

---

7. Authentication & Authorisation

Provider: Emergent-managed Google OAuth

1. Frontend opens `https://auth.emergentagent.com/?redirect=<our-callback>`
2. User signs in with Google → Emergent issues a one-time `code`
3. Frontend posts code to `/api/auth/exchange` → backend exchanges with Emergent for `{ user_id, email, name }`
4. Backend either creates a new `users` row OR updates existing → issues a `session_token`, stores in `user_sessions`
5. Returns Set-Cookie (httpOnly, secure, sameSite=None) AND the token in JSON for mobile/curl use

Role assignment

- First user ever to sign in → bootstrapped as `manager`
- Email in `MANAGER_EMAILS` env var (comma-separated) → `manager`
- All others → `associate`

Dependencies (FastAPI)

- `get_current_user(request)` — extracts session_token from cookie OR `Authorization: Bearer …`, looks up `user_sessions`, returns `User` model. Raises 401 if invalid/expired.
- `require_manager(user = Depends(get_current_user))` — additionally checks `role == 'manager'`. Raises 403 if not.

Test session bypass (for QA / testing agent)

Two seeded test sessions in `user_sessions` with hardcoded tokens (in `/app/memory/test_credentials.md`) so the testing agent can hit endpoints without OAuth round-trips.

---

8. Backend Modules & API Surface

Module map

File	LoC	Purpose
`server.py`	3,400	Main FastAPI app, auth, BI proxy, dashboards, NBA, segments, campaigns, customer sync, audit. Several routers inlined to circumvent prior deployment-pipeline issues.
`loyalty.py`	1,040	Tier qualification, voucher engine, distribution analytics, voucher-cost calculator
`routes/duplicates.py`	240	Phone-only duplicate detection with cached background scan
`routes/social.py`	800	11 social-listening endpoints (FB integration, sentiment classifier, auto-tasks)
`routes/insights.py`	500	Manager KPI dashboards, attribution, cohort analytics
`routes/whatsapp.py`	200	WhatsApp BSP scaffold (mock provider)
`routes/training.py`	150	Associate training/sandbox mode
`routes/push.py`	300	Mobile push composer + dispatcher
`sms.py`	80	Mock SMS provider; pluggable real BSP later
`facebook_sync.py`	250	FB Graph API pulls (pages, posts, reviews)
`fcm_worker.py`	200	Firebase Cloud Messaging dispatcher (lazy-loaded)

Endpoint surface (high-level groups)

Auth (`/api/auth/*`)

- `POST /exchange` — Google OAuth code → session
- `GET /me` — current user from session
- `POST /logout`

BI proxy (`/api/bi/*`) — read-through cache with walk-in filter

- `GET /kpis` · `/sales-summary` · `/country-summary` · `/daily-trend`
- `GET /top-customers` · `/customer-search` · `/customer/{id}` · `/customer/{id}/products`

- `GET /churned-customers` · `/orders` · `/inventory` · `/top-skus` · `/customer-frequency`
- `GET /locations`

Customer clienteling

- `GET /customers/grid` — paginated 163k customer view with filters
- `GET /customers/freshness` — cache age + sync job status
- `POST /customers/{id}/forget` — Kenya DPA right-to-be-forgotten
- `POST /customers/{id}/notes` · `/tasks` · `/preferences` · `/messages` · `/consent`
- `GET /customers/duplicates` · `POST /customers/duplicates/rescan`

Sync (admin)

- `POST /admin/customer-cache/full-sync` · `GET /admin/customer-cache/full-sync/status`
- `POST /customers/sync/full` · `GET /customers/sync/status` (inlined alias)
- `POST /customers/sync/incremental`
- `GET /admin/customer-cache/export` (manager-only, JSONL stream)

Dashboards (`/api/dashboard/*`)

- `GET /today` — associate home (KPIs + follow-ups + top VIPs + win-back)
- `GET /call-list?with_nba=true` — AI-prioritised customer outreach queue
- `GET /manager` — manager KPI strip
- `GET /attribution?days=30` — clienteling → revenue attribution

Loyalty (`/api/loyalty/*`) — see §10

- `GET /config` · `PUT /config`
- `POST /recompute` — kicks off background tier refresh
- `GET /customer/{id}` — tier, progress, retention, voucher, benefits
- `GET /distribution?date_from&date_to` — tier counts + per-tier sales/AOV/freq
- `GET /voucher-cost?date_from&date_to&redemption_rate=0.5`
- `GET /approaching-upgrade?within_kes=25000`
- `GET /anniversary-queue?within_days=30`
- `GET /pulse`
- `POST /customer/{id}/voucher/issue` · `POST /voucher/{id}/redeem`
- `POST /customer/{id}/styling/book`
- `GET /vouchers` · `/vouchers/expiring` · `/audit`

Social (`/api/social/*`)

- `GET /posts` · `/feedback` · `/mentions` · `/influencers` · `/dms`
- `GET /customer/{id}/timeline`

- `POST /handles/link` · `POST /reply`
- `POST /classify` — Claude Sonnet sentiment + themes
- `POST /facebook/discover` · `GET /facebook/pages` · `POST /facebook/sync`
- `POST /auto-tasks/run` · `GET /auto-tasks` · `/runs` · `/kpi`

Templates & messaging

- `GET /templates` · `POST /templates` · `PUT /{id}` · `DELETE /{id}`
- `POST /campaigns/preview` · `POST /campaigns/send`
- `POST /segments/preview` · `GET /segments/filters`

Insights & cohorts

- `GET /insights/cohorts` · `/insights/retention` · `/insights/cohort/{id}`
- `GET /insights/template-performance`
- `GET /bi/return-rate-trend` · `/bi/channel-attribution`

Lookbooks

- `POST /lookbooks` · `GET /{id}` · `PUT /{id}/items` · `DELETE /{id}`
- `GET /public/lookbooks/{token}` (no auth)
- `POST /public/lookbooks/{token}/interest`

System

- `GET /health` · `GET /audit`

---

9. Data Pipelines & Sync Strategy

Sources of truth

Data	Source	Refresh cadence	Cache TTL
Customer profiles + lifetime sales	Vivo BI `/top-customers`, `/customer-search`	On-demand + nightly	In-memory 60s
Order line items	Vivo BI `/orders`	On-demand	In-memory 60s
Rolling 12-mo spend per customer	Computed in CRM from BI `/top-customers` 4× quarterly	Nightly 23:00 UTC	Persistent (`customer_cache.spend_12mo_kes`)
RFM tier	Computed in CRM	On every cache write	Persistent
Loyalty tier	Computed from spend_12mo + recency	Nightly + on-demand	Persistent
Facebook posts/reviews	FB Graph v19	Every 15 min	Persistent

Customer cache full sync (~6 min)

1. Walk 26 calendar quarters from 2020-Q1 → today
2. For each quarter, fan out to Kenya / Rwanda / Uganda in-store countries (`/top-customers?country=X&date_from=…&date_to=…&limit=50000`)
3. Drop Safari / Zetu / Holding / Online channels at filter step
4. Drop walk-in / staff-placeholder records (`is_walkin_record()`)
5. Upsert via `pymongo.UpdateOne` with `bulk_write` in chunks of 2,000 (Atlas-friendly, retried 3× on timeout)
6. After all quarters: recompute rolling 12-mo spend + tier for every customer
7. Update `sync_jobs._id='customer_cache_full_sync'` with progress; UI polls every 5s

Nightly incremental sync (23:00 UTC = 02:00 Africa/Nairobi)

- Refreshes only the current + previous quarter (~30s)
- Loyalty `_refresh_spend_12mo()` runs after — pulls last-365-days top-customers, recomputes tiers, audits upgrades/demotions
- Birthday voucher auto-issue runs alongside (one per non-dormant member per anniversary day in the current month)

Boot-time auto-trigger

- On startup (after 60s delay to let K8s readiness probe pass):
- If `customer_cache.count() < 100`: pull 2,000 top-customers quickly
- If `customer_cache.count() < 100,000`: trigger full background sync
- Stale running jobs (>5 min without `updated_at`) are treated as orphaned and re-run

Walk-in filter rule (data hygiene)

```
def is_walkin_record(profile):
    # Name contains \bwalk(?:in|n|s|ed|ing)?\b — whole word
    # OR email local-part starts with 'vivo'
    # OR email domain contains 'vivofashion'
    # Preserves Walker surnames and @vivoenergy / @survivor.* accidents.
```

Applied at 4 BI read endpoints AND inside `_cache_customers()` so future syncs can't re-introduce walk-ins.

---

10. Loyalty Programme Engine

Tier qualification (current production config)

Tier	Qualify	Retain	Voucher	Discount
Dormant	No purchase in last 365 days	—	—	—
Bronze	Any first purchase, ≤ KES 100,000 in 12mo	1 purchase / 12mo	KES 2,500	0%
Silver	> KES 100,000 in 12mo	KES 70,000 in 12mo	KES 5,000	5% on full-price
Gold	> KES 250,000 in 12mo	KES 150,000 in 12mo	KES 10,000	10% on full-price

> Note: a percentile-based redesign (Silver KES 25k / Gold KES 60k → 80/15/5 distribution) is modelled and ready in `scripts/tier_threshold_and_points_cost.py` but not yet activated in `loyalty_config`.

Refresh algorithm — `_refresh_spend_12mo()`

1. Build 4 ~91-day windows working back from today (covers full 365-day rolling window without hitting the 50k BI cap)
2. Per window, fan out parallel calls to Kenya / Rwanda / Uganda (in-store only; Online + Safari excluded by definition)
3. Sum per-customer total_sales across windows

4. Clamp negative aggregates to 0 (returns > purchases in a quarter shouldn't mis-tier)
5. Classify tier; capture audit event on upgrade/demotion
6. Bulk write to `customer_cache.spend_12mo_kes` + `loyalty_tier`
7. Customers NOT in the agg map but with `last_purchase_date >= 365 days ago` → Bronze with 0 spend
8. Customers with `last_purchase_date < 365 days ago` OR null → Dormant

Voucher engine

- Issue: birthday vouchers auto-issue at start of birthday month for non-dormant members
- Status: `issued` → `redeemed` (manual via `POST /voucher/{id}/redeem`) OR `expired` (after `voucher_validity_days`)
- Cost projection: `GET /loyalty/voucher-cost?date_from=&date_to=&redemption_rate=0.5`

Drill-down analytics (UI)

- Clickable tier cards → modal listing every customer in that tier (paginated, CSV export)
- Date-filtered "Sales in window" + "Rolling 12-mo spend" + "AOV" + "Frequency" per tier
- Estimated discounts received per tier (Silver/Gold only)

Back-test scripts (in `/app/backend/scripts/`)

- `simulate_2025_loyalty.py` — what would the 5/10% discount programme have cost in 2025? KES 13.5M
- `simulate_2025_vouchers.py` — birthday voucher liability: KES 65.6M expected @50% redemption
- `sim_flat_points_2025.py` · `sim_hybrid_*.py` — points-programme alternatives modelled against real BI data
- `tier_threshold_and_points_cost.py` — percentile-based threshold redesign + points cost across 3 scenarios

---

11. Frontend Application

Routing tree (React Router v7)

```
/                       redirect to /today (if authed) or /login
/login                  Emergent Google sign-in
/today                  Associate home (call list, KPIs, top VIPs)
/customers              Paginated grid (163k rows)
/customers/:id          360 profile (tabs: Purchases / Preferences / Notes / Follow-ups / Messages / Social
/inbox                  Social feedback + DM triage
/lookbooks              Lookbook builder
/lookbooks/:id          Lookbook editor
/share/:token           Public lookbook view (no auth)
/insights               Manager dashboards (Sales / Country / Top stores / Social)
/insights/cohorts       Cohort retention heatmap
/loyalty                Tier dashboard + drill-downs
/loyalty/app-preview    Mobile member-app preview iframe
/operations             Audit log, sync status, freshness
/settings               Templates, Facebook, sync, push composer
```

Design system

- Theme aligned with bi.vivofashionbrands.com
- Forest green primary `#0F4D31` (--vivo-navy)
- Warm orange accent `#ED7C2A`
- Cream background `#FCEFD9` (--vivo-bg)
- Gold border `#C9A961` (--vivo-gold)
- Fonts: `Outfit` (display) + `Manrope` / `Inter` (body)
- Components: shadcn/ui from `/components/ui/`
- Icons: lucide-react (NO emoji per design guidelines)
- Animation: micro-interactions on hover, staggered card reveals, `press-effect` utility for buttons
- Spacing: 2-3× more whitespace than feels comfortable

Critical components

Component	Purpose
`AppShell`	Top nav, sidebar, search, notifications bell, user menu
`CustomerSearch`	Header autocomplete (BI customer-search debounced)
`LoyaltyBadge`	Inline tier pill (Bronze/Silver/Gold/Dormant)
`RFMBadge`	Inline RFM pill (vip/loyal/promising/at_risk/churned/new)
`TierCustomersModal`	Drill-down list from Loyalty card click
`VoucherCostPanel`	Birthday-voucher cost calculator with date presets
`FreshnessBadge`	Shows "Last synced X min ago" + manual refresh
`CustomerSyncCard`	Live progress of full sync (cached / unique / quarters / status)
`PushComposer`	Manager → mobile-app segmented push notification
`NbaCard`	AI Next-Best-Action with "Use this script" button

State management

- React local state + `useEffect` data fetching
- No Redux / Zustand — kept lean
- `axios.create({ baseURL: process.env.REACT_APP_BACKEND_URL + '/api', withCredentials: true })`

---

12. AI Integrations

Emergent LLM key

A single universal key (`EMERGENT_LLM_KEY` in `backend/.env`) routes Claude, OpenAI, and Gemini calls through Emergent's billing wrapper. No direct SDK keys.

Claude Sonnet 4.5 — Three production use cases

1. NBA (Next-Best-Action)

Endpoint: `GET /api/customers/{id}/nba` Input: tier, RFM, last 5 orders, preferences, last contact date, anniversary status Output: `{ action, why, script, urgency }` Cache: 6 hours per customer Fired by: customer profile page load + dashboard call-list (background precompute, capped at 12/call)

2. Sentiment classifier (social listening)

Endpoint: `POST /api/social/classify` Input: post/feedback text Output: `{ sentiment: positive|neutral|negative, themes: [from fixed vocab] }` Used in: Inbox triage, weekly auto-task generation

3. Negative-theme auto-tasks

Schedule: Every Monday (or manual "Run now") Logic: Cluster last-7-day negative feedback by theme → create one follow-up task per qualifying theme for every manager Idempotent: per ISO week — re-running is safe

Cost monitoring

- Emergent LLM key budget visible in Profile → Universal Key
- NBA cache + 6h TTL keeps Claude calls under ~50/day in steady state

---

13. External Integrations

Vivo BI API (BigQuery-backed)

- Base: `https://vivo-bi-api-666430550422.europe-west1.run.app`
- No auth (private network)
- Endpoints used: `/kpis`, `/sales-summary`, `/country-summary`, `/daily-trend`, `/top-customers`, `/customer-search`, `/customer-products`, `/churned-customers`, `/orders`, `/inventory`, `/top-skus`, `/customer-frequency`, `/locations`
- Retry policy: 3× with exponential backoff on 503

Facebook Graph API v19

- App credentials in `backend/.env`: `FACEBOOK_APP_ID`, `FACEBOOK_APP_SECRET`, `FACEBOOK_CLIENT_TOKEN`
- Per-page admin tokens stored in `facebook_pages` collection after user pastes `user_access_token` to `POST /api/social/facebook/discover`
- Sync job pulls `/posts`, `/comments`, `/ratings` per linked page every 15 min
- Token rotation: when a token expires (code 190), the page is marked `needs_reauth` and surfaced in Settings

Emergent Managed Google OAuth

- Flow: code-exchange via `POST /api/auth/exchange`
- No client secrets in our code — Emergent handles the Google OAuth dance

WhatsApp / SMS BSP (scaffolded, mock)

- `sms.py` returns `delivery_status: "mock_delivered"` in v1
- Pluggable: env vars `BSP_PROVIDER`, `BSP_API_KEY` reserved for Africa's Talking / Twilio

Firebase Cloud Messaging (mobile push, off by default)

- `fcm_worker.py` lazy-loads firebase-admin only if `FIREBASE_SERVICE_ACCOUNT_JSON` env present
- Otherwise dispatcher returns `mock_sent`

MongoDB Atlas (production)

- Cluster: `customer-apps-shard-XX-XX.9dszfg.mongodb.net`
- DB: `vivo_crm`
- All connections via `MONGO_URL` env var (not hardcoded — enforced platform policy)

---

14. Data Quality & Hygiene

Duplicate detection — Phone-only (per business owner)

- Rule: last 9 digits of phone number match → group as duplicates
- Email and name matching explicitly REMOVED (too many false positives)
- Background scan walks 163k customers in <60s, materialises result in `duplicates_cache`
- UI: `/settings` → Duplicates tab; merge action is planned (P2)

Walk-in / staff-placeholder exclusion

- Rule (option b, agreed Feb 2026):
- Name `\bwalk(?:in|n|s|ed|ing)?\b` whole-word
- OR email local-part starts with `vivo`
- OR email domain contains `vivofashion`
- Applied at: 4 BI read endpoints + `_cache_customers()` write path
- Result: 38 placeholder rows purged; 8 genuine Walker surnames preserved

Online channel exclusion

- Strict in-store filter: only `Kenya / Rwanda / Uganda` with channel NOT containing `safari`, `zetu`, `online`, `holding`, `zoya`
- Applied to all loyalty calculations, tier metrics, and exports
- The 0.04% "Guest" walk-in revenue (KES 488k in 2025) is negligible

Audit log

- Every manager action that touches customer data is logged: `audit_log.event_id` + actor + action + entity + at
- Searchable from `/operations` page
- Required for Kenya DPA compliance

---

15. Mobile App (React Native / Expo)

Purpose

Member-facing loyalty companion. Customers see their tier, points/spend toward next upgrade, vouchers, anniversary status, and receive segmented push notifications.

Key screens

- `app/(auth)/sign-in.tsx` — phone OTP (mock `000000` in dev)
- `app/(tabs)/home.tsx` — tier badge, 12-mo spend ring, next-tier progress
- `app/(tabs)/vouchers.tsx` — active + redeemed vouchers
- `app/(tabs)/account.tsx` — preferences, opt-in toggles

Backend touchpoints

- `POST /api/mobile/auth/otp/start` · `POST /api/mobile/auth/otp/verify`
- `GET /api/loyalty/customer/{id}` (same endpoint as CRM)
- `POST /api/mobile/push/register-device` (FCM token storage)

Distribution

- iOS: TestFlight → App Store
- Android: APK via EAS Build (job artifact in EAS dashboard)
- Web preview: `/loyalty/app-preview` iframe inside CRM

---

16. Security & Compliance (Kenya DPA)

Consent management

- Per-channel (WhatsApp / SMS / Email) opt-in capture on customer profile

- History stored: `consent.history[]` with timestamp + actor
- All send endpoints check consent BEFORE dispatch; blocked sends are logged in `audit_log`

Right to be forgotten

- `POST /api/customers/{id}/forget` (manager only, exact-name confirmation in UI)
- Anonymises: notes, tasks, messages, lookbooks, preferences, NBA cache, customer_cache, social handles
- Flips all consent to opted-out
- Creates a tombstone in `forget_log` for audit (we keep the ID + at + actor, but no PII)

Data export lockdown

- `GET /api/admin/customer-cache/export` requires `Depends(require_manager)`
- Unauthenticated → 401, Associate → 403, Manager → 200 streams 79 MB JSONL
- Previously public during Atlas migration; locked down May 2026

PII at rest

- MongoDB Atlas encryption-at-rest enabled by default
- No PII in application logs (we log `customer_id` only, never name/phone/email)
- Environment variables (MONGO_URL, EMERGENT_LLM_KEY) NEVER in source code; loaded from `.env` per platform policy

CORS

- `backend/.env`: `CORS_ORIGINS="*"` (permissive; tightened in production via Cloudflare WAF rules)
- All API responses include `Access-Control-Allow-Credentials: true` for cookie auth

---

17. Deployment & Infrastructure

Environments

Env	URL	Database	Purpose
Preview / dev	`crm-platform-145.preview.emergentagent.com`	Local MongoDB	Development & testing
Production	`crmvivofashiongroup.com`	MongoDB Atlas	Live deployment

Build pipeline (Emergent native deployment)

1. Source repo committed via "Save to GitHub"
2. Emergent build pipeline analyses `/app/` → detects FastAPI + React + Expo
3. Generates supervisord.conf with `[program:backend]`, `[program:frontend]`, `[program:expo]`
4. Builds React static bundle (`yarn build`)
5. Containerises with backend + static frontend + supervisord
6. Deploys to K8s cluster
7. Cloudflare-for-SaaS routes custom hostname to the deployment

Environment variables (production)

- `MONGO_URL` — Atlas connection string (set in Emergent deployment secrets)
- `DB_NAME` — `vivo_crm`
- `EMERGENT_LLM_KEY` — Universal LLM key
- `FACEBOOK_APP_ID`, `FACEBOOK_APP_SECRET`, `FACEBOOK_CLIENT_TOKEN`
- `MANAGER_EMAILS` — comma-separated manager allow-list
- `REACT_APP_BACKEND_URL` (frontend) — set to `https://crmvivofashiongroup.com`

Startup hardening (May 2026 deployment fix)

- Deferred customer-cache warm: `await asyncio.sleep(60)` so K8s readiness probe (30s) passes first
- First scheduler tick delayed from 20s → 120s after boot
- Loyalty refresh wrapped in `asyncio.wait_for(..., timeout=240)` to avoid hanging the scheduler thread on Atlas latency

Custom hostname activation

- DNS: A records for `crmvivofashiongroup.com` → `162.159.142.117` and `172.66.2.113` (Cloudflare-for-SaaS edge)
- Activation: Emergent deployment "Entri" button OR support@emergent.sh
- Error 1034 indicates Custom Hostname not yet registered; not a code issue

---

18. Operations & Monitoring

Health endpoints

- `GET /api/health` → `{ status, bi_api_reachable, time }`
- Manager dashboard shows: customer cache size, last sync time, sync job status, BI reachability

Sync visibility

- `/operations` page (manager) shows:
- Customer cache row count vs BI total
- Last full-sync completion time + duration
- Last incremental sync time
- Last loyalty refresh + upgrades/demotions count
- Last duplicate scan + groups found
- Facebook pages connected + last successful sync per page

Audit trail

- Every privileged action logged with actor + entity + payload + IP + UA + timestamp
- Filterable by user, action type, date range
- Retention: indefinite (Kenya DPA 7-year requirement)

Logs

- Backend: `/var/log/supervisor/backend.{err,out}.log`
- Frontend: `/var/log/supervisor/frontend.{err,out}.log`
- Expo: `/var/log/supervisor/expo.{err,out}.log`
- Application: structured via Python `logging` module with `vivo` logger namespace

Backups

- Atlas: automated daily snapshots (Atlas-managed)
- Manual export: `GET /api/admin/customer-cache/export?fmt=jsonl` (manager-only)

---

19. Development Workflow

Local development

1. Backend hot-reload: uvicorn with `--reload`, watched by supervisord
2. Frontend hot-reload: CRACO dev server on `:3000`
3. MongoDB: local supervisord-managed instance
4. Env: `backend/.env`, `frontend/.env`, `mobile/.env` (all in repo, with placeholders for secrets)

Testing

- Backend: pytest in `/app/backend/tests/` (19/19 green as of iteration 6)
- Frontend: Playwright via `testing_agent_v3_fork` (regression suite covers 80+ flows)
- Manual: `curl` with `Authorization: Bearer test_session_vivo_mgr_…` (test sessions in test_credentials.md)

Commit flow

- Git managed via Emergent "Save to GitHub" feature (manual or auto-commit)
- Preserve `.git` and `.emergent` folders
- Branch strategy: single main branch + preview deployments per commit

Deployment

1. Develop in preview env
2. Test via testing_agent or manually
3. Click "Deploy" in Emergent dashboard
4. Build pipeline produces production container
5. Cloudflare routes traffic to new container

---

20. Known Limitations & Backlog

Currently mocked / scaffolded

- WhatsApp BSP: returns `delivery_status: "mock_delivered"` — real Africa's Talking / Twilio integration pending
- SMS provider: same — mocked
- BigQuery social tables: 180 mocked feedback items — real schemas pending from Vivo data team
- Mobile push (FCM): lazy-loaded; works in mock mode unless `FIREBASE_SERVICE_ACCOUNT_JSON` env set

Production deployment gotchas

- Custom hostname registration may need manual support intervention (Error 1034)
- Atlas connection sometimes slow on first call after deploy — hardened with 60s startup delay + 240s timeouts on schedulers
- Production deployment pipeline has historically been slow to pick up new route files — workaround: inline critical routes into `server.py`

Backlog (prioritised)

- P1: Real WhatsApp BSP (Africa's Talking or Twilio)
- P1: Customer ↔ associate assignment (currently any associate sees any customer)
- P1: Inbound message webhooks → message timeline
- P1: Real BigQuery social tables
- P1: Native Mongo datetimes (replace ISO-string timestamps)
- P2: server.py chunking (extract AI, inbox, segments into `/app/backend/routes/`)
- P2: Background social classifier with in-process lock instead of inline-on-read
- P2: Redis or cachetools.TTLCache instead of in-process BI cache
- P2: Drag-reorder + real product images on lookbooks (needs BI feed URLs)
- P2: Lifespan handlers instead of `@app.on_event` (FastAPI deprecation)
- P2: "Merge group → primary customer" action on duplicates page
- P3: Points-programme engine (ledger collection, earn/redeem APIs) — design complete in scripts/
- P3: Real Meta / TikTok / X post-fetcher
- P3: Optional Swahili translation pass
- P3: Odoo 18 POS connector (codebase scaffolded in `/app/odoo/`)

Appendix A — Critical configuration files

`backend/.env` (template, no secrets shown)

```
MONGO_URL=mongodb://localhost:27017
DB_NAME=test_database
EMERGENT_LLM_KEY=<from-emergent>
FACEBOOK_APP_ID=
FACEBOOK_APP_SECRET=
FACEBOOK_CLIENT_TOKEN=
MANAGER_EMAILS=
FACEBOOK_SYNC_INTERVAL_MIN=15
CORS_ORIGINS=*
```

`frontend/.env`

```
REACT_APP_BACKEND_URL=https://crm-platform-145.preview.emergentagent.com
```

`/etc/supervisor/conf.d/supervisord.conf`

```
[program:backend]
command=/root/.venv/bin/uvicorn server:app --host 0.0.0.0 --port 8001 --workers 1 --reload
directory=/app/backend
…

[program:frontend]
command=yarn start
environment=HOST="0.0.0.0",PORT="3000"
directory=/app/frontend
…

[program:mongodb]
command=/usr/bin/mongod --bind_ip_all
…

[program:expo]
command=yarn expo start --tunnel --port 3000
directory=/app/mobile
…
```

Appendix B — Key business rules summary

1. In-store only: All loyalty + reporting metrics filter to Kenya / Rwanda / Uganda physical stores. Safari (was Online), Shop Zetu, and Holding-location channels are explicitly excluded.
2. Net spend, not gross: Tier qualification uses `total_sales_kes` from BI (net of order-line discount, exc VAT). Returns subtract.
3. Rolling 12-month window: Tier qualification is rolling (today − 365 days → today), recomputed nightly. A customer can move tiers up or down within a year.
4. Phone-only duplicate matching: Per business owner (May 2026 decision). Email and name matching too noisy.
5. Walk-in exclusion: Whole-word "walk" in name OR email local-part starts with "vivo" OR email domain contains "vivofashion". Preserves Walker surnames and accidental "vivo" substrings.
6. Birthday vouchers, not signup bonuses: Vouchers issue at the start of birthday month for non-dormant members; dormant customers don't get vouchers (re-activation is a separate campaign).
7. Discount on full-price only: 5% Silver / 10% Gold applies only to line items where `discount_kes == 0` AND `total_sales_kes > 0`. Sale items don't stack.

Appendix C — Glossary

Term	Definition
BI	Vivo's BigQuery-backed Business Intelligence API
RFM	Recency / Frequency / Monetary — segmentation framework
NBA	Next-Best-Action — AI-suggested associate action per customer
BSP	Business Solution Provider (for WhatsApp / SMS)
FCM	Firebase Cloud Messaging (mobile push)
DPA	Kenya Data Protection Act 2019
AOV	Average Order Value
Clienteling	Personalised one-to-one selling via tracked outreach
Cohort	Group of customers who made first purchase in same month
RFM tier	`vip` / `loyal` / `promising` / `at_risk` / `churned` / `new`
Loyalty tier	`dormant` / `bronze` / `silver` / `gold`

End of document.

For questions or updates, modify this file at `/app/memory/VIVO_CRM_ARCHITECTURE.md`.

The PRD changelog at `/app/memory/PRD.md` tracks all feature additions chronologically.